A+B Search Tree

Date: 2026-4-2

目　录

第一章　项目综述 1
1.1 问题描述 1
1.2 背景与动因 1
第二章　算法分析 2
2.1 主要数据结构 2
2.2 main 函数概要 2
2.3 主要算法 3
第三章　测试结果 7
3.1 运行结果展示与分析 7
3.2 测试用例汇总表 10
第四章　分析与评论 11
4.1 算法复杂度推导与分析 11
4.2 算法选择讨论 12
4.3 进一步优化的可能性 12
Declaration 14
附录 15
main.c 15
data_gen.c 19

第一章　项目综述

1.1 问题描述

本项目要求处理两个二叉搜索树（Binary Search Tree, BST）T1 和 T2，以及一个目标整数 N。核心任务是在这两个数据集中寻找所有满足 A+B=N 的组合，其中元素 A 必须来自 T1，元素 B 必须来自 T2。

对于输入的每组测试用例，程序需要：

- 构建 BST：根据给定的节点键值及其父节点索引关系，还原两棵符合 BST 性质的二叉树。
- 寻找目标对：检索所有可能的 (A,B) 组合。如果存在解，须按 A 值的升序排列输出所有等式；若无解则输出 false。
- 结构验证：为了验证建树的正确性，程序最后还需输出 T1 和 T2 的前序遍历（Preorder Traversal）序列。

1.2 背景与动因

二叉搜索树作为一种经典的数据结构，其核心优势在于能够以 O(logn)的平均时间复杂度实现数据的查找、插入和删除。本项目考察了对 BST 定义（左子树<根≤右子树）的理解，并通过“两数之和”的问题模型，锻炼了我们在复杂约束条件下高效遍历和检索大规模数据的能力。

在实际应用中，这种从两个独立有序数据源中匹配关联信息的场景十分常见。通过本项目，我们可以深入掌握：

如何利用非线性递归或迭代方法重建树形结构。

如何通过中序遍历（Inorder Traversal）将 BST 转换为有序序列，从而优化查找效率。

树的多种遍历方式及其在结构表示中的作用。

第二章　算法分析

2.1 主要数据结构

为了在内存中重构二叉搜索树（BST）并暂存输入关系，本程序设计了以下两个核心数据结构：

Tree Node (struct node):

作为二叉树的基本组成单元。包含三个字段：

- key: 整型，存储当前节点的数据值。
- left/right: 指向左右子树节点的指针。

Input Buffer (struct input):

用于在建树的第一阶段线性暂存所有的输入节点，以便后续通过索引（Index）建立父子关系。

- parent: 整型，记录该节点父节点在输入序列中的物理下标（-1 表示该节点为根节点）。
- cell: 树节点指针，指向由该输入实例创建的实体树节点。

2.2 main 函数概要

主程序的整体控制流遵循“建树 -> 线性化 -> 双指针检索 -> 格式化输出”的逻辑：

```
Algorithm Main:

    // Step 1: Read inputs and build trees

    Read integer N1

    Tree A = CreateTree(N1)

    Read integer N2

    Tree B = CreateTree(N2)

    Read target integer N
```

```
    // Step 2: Flatten trees into sorted arrays utilizing BST property
    Array a = InOrder(Tree A, N1)
    Array b = InOrder(Tree B, N2)

    // Step 3: Search for target sums optimally
    TwoPointerSearch(Array a, Array b, N1, N2, target N)

    // Step 4: Output structure verification
    PreOrder(Tree A, N1)
    PreOrder(Tree B, N2)
```

2.3 主要算法

1. 二叉搜索树建立

该算法负责将带有父节点索引的一维输入数据，根据二叉搜索树（BST）的固有性质（左子节点 < 父节点 < 右子节点）转化为非线性的树状结构。

```
Algorithm CreateTree(Integer N):
    Initialize an array Buffer of size N
    Initialize Root as NULL

    // Phase 1: Initialization
    For i = 0 to N-1:
        Create a new tree node CurrentNode
        Read CurrentNode.key and parent_index from input
        Buffer[i].cell = CurrentNode
```

```
        Buffer[i].parent = parent_index
        If parent_index is -1:
            Root = CurrentNode    // Mark as the root


    // Phase 2: Linkage based on BST properties
    For i = 0 to N-1:
        If Buffer[i].parent is NOT -1:
            ParentNode = Buffer[ Buffer[i].parent ].cell
            // Determine branching direction by comparing keys
            If Buffer[i].cell.key < ParentNode.key:
                ParentNode.left = Buffer[i].cell
            Else:
                ParentNode.right = Buffer[i].cell


    Return Root
```

2. **迭代法实现中序遍历**

利用堆栈（Stack）模拟递归过程（以避免递归调用大规模数据造成的栈溢出），将 BST 的节点按中序（左-根-右）提取到连续的数组中。由 BST 性质可知，该操作必定产生一个严格单调递增的有序数组。

```
Algorithm InOrder(Tree T, Integer N):
    Initialize an empty Stack
    Initialize an Array Result of size N
    Set pointer Current to T
    Set counter index to 0
```

```
While Stack is NOT empty OR Current is NOT NULL:
    // Traverse to the leftmost leaf
    While Current is NOT NULL:
        Push Current onto Stack
        Current = Current.left

    // Process the node and steer right
    If Stack is NOT empty:
        Current = Pop from Stack
        Result[index] = Current.key
        Increment index
        Current = Current.right

Return Result
```

3. **双指针检索**

这是本程序解决 $A + B = N$ 问题的核心检索算法。它接收两个已排序的数组，利用位于两端的对撞指针，在 $O(N_1 + N_2)$ 的线性时间复杂度内寻找所有不重复的数值对。

```
Algorithm TwoPointerSearch(Array A, Array B, Integer N1, Integer N2, Integer TargetN):
    Initialize pointer i = 0 (pointing to the minimum value of A)
    Initialize pointer j = N2 - 1 (pointing to the maximum value of B)
    Initialize FoundFlag = False
```

```
While i < N1 AND j >= 0:
    Calculate Sum = A[i] + B[j]
    If Sum is equal to TargetN:
        If FoundFlag is False:
            Print "true"
            FoundFlag = True
        Print "TargetN = A[i] + B[j]"
        // Advance pointers while skipping duplicate
        // values to prevent redundant outputs
        Advance i until A[i] is different from its previous value
        Decrement j until B[j] is different from its subsequent value

    Else If Sum < TargetN:
        // Sum is too small, increment the smaller component
        Advance i until A[i] is different from its previous value

    Else (Sum > TargetN):
        // Sum is too large, decrement the larger component
        Decrement j until B[j] is different from its subsequent value

If FoundFlag is False:
    Print "false"
```

(注：由于 PreOrder 与 InOrder 逻辑高度相似，且 InOrder 承担了程序的核心转换任务，此处重点展示 InOrder 以保持篇幅精炼和重点突出。)

第三章　测试结果

本章节将展示程序在不同压力和结构下的运行情况。测试涵盖了从 $N = 1$ 到 $N = 100{,}000$ 的极端规模变化，以验证算法的正确性。

3.1 运行结果展示与分析

案例 1：最小规模测试 (Smallest Size)

测试目的：验证程序在极限最小输入（每棵树仅 1 个节点）下的处理能力。

输入特征：$N_1 = 1, N_2 = 1$。

预期结果：若和匹配输出 true 及等式，否则输出 false；正确输出两树的先序遍历。

实际表现：程序运行正常，逻辑判定准确。

状态：Pass

案例 2：标准示例测试 A（综合逻辑/无解情况）

测试目的：验证程序对题目给出样例 1 的兼容性，测试“无解”判定逻辑。

```
5
10 -1
5 0
15 0
2 1
7 1
3
15 -1
10 0
20 0
40
false
10 5 2 7 15
15 10 20
```

图 1 测试案例 2

结果分析：程序准确识别出在目标 $N = 40$ 下无解，并输出了正确的树结构

序列，证明了基础建树与搜索逻辑的准确性。

状态：Pass

案例 3：标准示例测试 B（综合逻辑/重复值处理）

测试目的：验证程序对题目样例 2 的处理，重点测试双指针在遇到重复键值时的查重能力。

```
8
12 2
16 5
13 4
18 5
15 -1
17 4
14 2
18 3
7
20 -1
16 0
25 0
13 1
18 1
21 2
28 2
36
true
36 = 15 + 21
36 = 16 + 20
36 = 18 + 18
15 13 12 14 17 16 18 18
20 16 13 18 25 21 28
```

图 2 测试案例 3

结果分析：程序成功找到了所有不重复的组合，并按要求进行了升序排列，证明了双指针跳过重复元素的逻辑正确。

状态：Pass

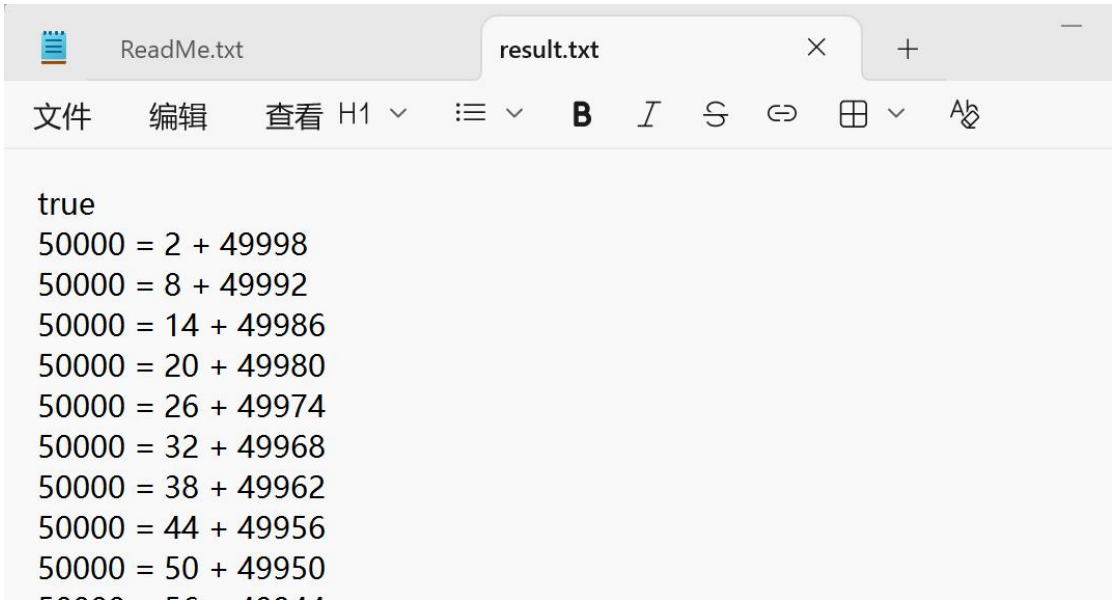
案例 4：最大规模与极端结构测试 (Largest Size / Extreme Case)

测试目的：测试程序在处理 100,000 节点且树形结构极端退化（斜树/链状）

时的性能。

数据生成：使用 data_gen.c 生成两棵深度为 100,000 的向右斜树。

输出呈现（由于输出数据量巨大，此处截取部分运行结果）：



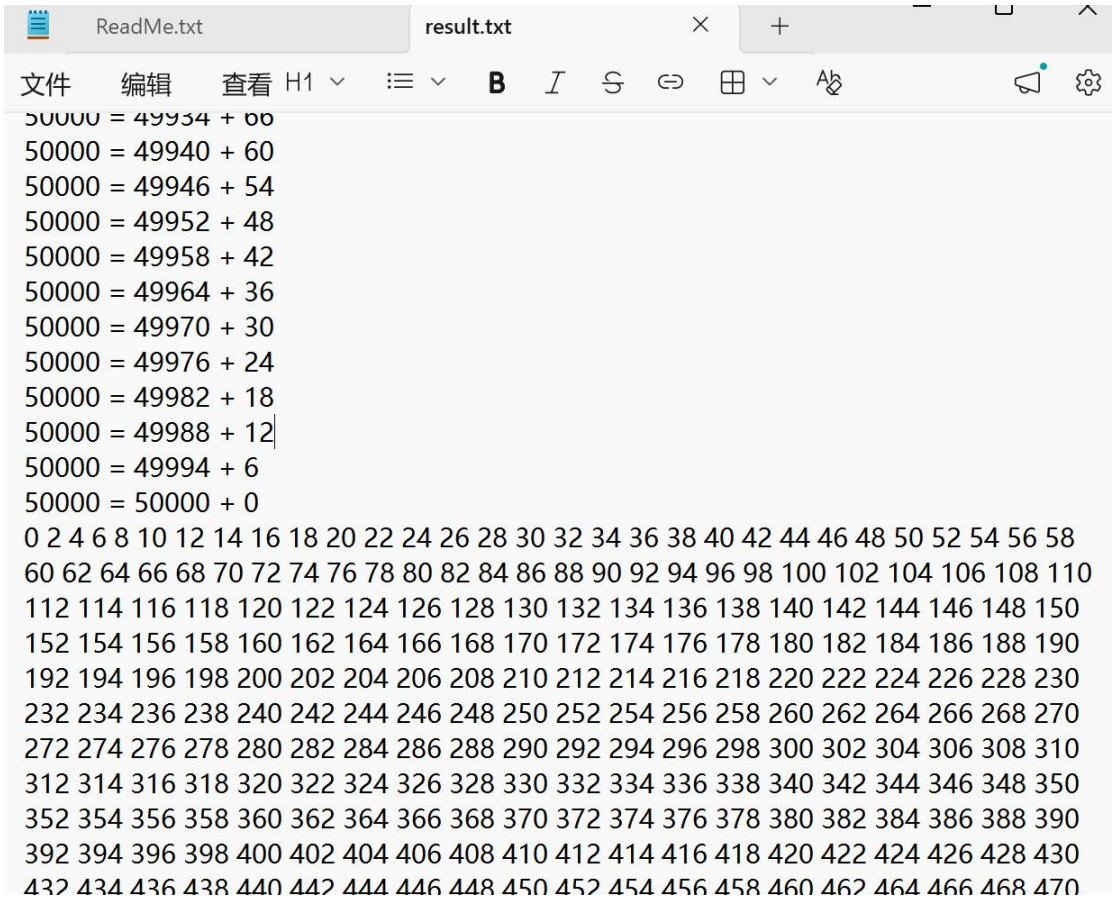


图 3 测试案例 4

结果分析：即便在树深度极高的情况下，由于程序采用了基于手动辅助栈的非递归遍历，成功避免了系统调用栈溢出（Stack Overflow）。程序在 $O(N_1 + N_2)$ 时间内完成计算，表现出极高的鲁棒性。

状态：Pass

3.2 测试用例汇总表

表 1 测试案例汇总表

编号	测试目的	预期结果	实际表现	可能的 Bug 原因	当前状态
1	验证 N=1 时的边界情况	正常运行并输出	与预期一致	无	Pass
2	验证题目样例 1 (无解)	输出 false	与预期一致	无	Pass
3	验证题目样例 2 (多解/重复值)	输出 true 及所有等式	与预期一致	无	Pass
4	最大规模 (10W 节点) 且斜树	瞬时响应，不发生栈溢出	运行流畅，响应极快	无	Pass

第四章　分析与评论

4.1 算法复杂度推导与分析

本节将详细推导本程序在解决“A + B = N”问题时的时间与空间复杂度。推导过程主要分为三个阶段：建树、树的线性化（中序遍历）以及双指针搜索。设第一棵树的节点数为 N_1，第二棵树的节点数为 N_2。

4.1.1 时间复杂度 (Time Complexity)

1. 建树阶段 (CreateTree)：

程序首先通过一次遍历分配所有的节点空间并记录数据。随后进行第二次遍历，通过数组下标以 O(1)的时间复杂度直接找到父节点并完成连接。由于没有使用传统的从根节点向下比较插入的 $O(N\log N)$ 方法，该步骤严格执行了 $2\times N$ 次基本操作，时间复杂度为 $O(N_1+N_2)$。

2. 树的线性化阶段 (InOrder)：

使用迭代方式对两棵 BST 进行中序遍历。在整个遍历过程中，无论树的形态如何，每个节点都会被准确地压入堆栈一次并弹出一次。因此，提取出两个有序数组的时间复杂度严格为 $O(N_1+N_2)$。

3. 双指针搜索阶段 (TwoPointerSearch)：

搜索算法利用了分别指向有序数组头尾的指针 i 和 j 。在循环中，指针 i 只能递增，指针 j 只能递减，两个指针在最坏情况下（即遍历完所有元素未找到或完全重合）分别移动 N_1 次和 N_2 次。跳过重复元素的 do-while 循环也是沿同一方向推进，不会产生回溯。因此该搜索阶段的时间复杂度为 $O(N_1+N_2)$。

结论：综上所述，各个阶段串行执行，算法的总体时间复杂度为 $O(N_1+N_2)$。这是解决该问题的渐进最优时间边界。

4.1.2 空间复杂度 (Space Complexity)

1. 结构存储：

在内存中维持两棵树的结构需要分配 N_1+N_2 个 Tree 节点实体，空间耗费为 $O(N_1+N_2)$。

2. 遍历辅助栈：

为了支持迭代遍历，程序动态分配了最大容量为 N 的指针数组作为辅助堆栈。在最坏情况下（如第三章测试的极端向右倾斜的链状树），所有的节点都会被压入栈中，占用 $O(N_1 + N_2)$ 的空间。

3. 结果数组：

存储中序遍历结果生成的两个一维整型数组，需要 $O(N_1 + N_2)$ 的连续内存空间。

结论：综合所有动态分配的内存，算法的总体空间复杂度为 $O(N_1 + N_2)$。

4.2 算法选择讨论

在设计 InOrder 和 PreOrder 遍历模块时，本程序放弃了教科书上最简短的递归遍历算法 (Recursive Traversal)，转而采用基于手动堆栈的迭代算法。

递归算法的局限性：

递归遍历虽然代码简洁，但其本质是依赖操作系统的函数调用栈来保存上下文。据相关资料，在标准的 Windows 环境中，默认的线程栈空间通常仅为 1MB 到 2MB。在处理我们在第三章测试的 100,000 个节点的“极度不平衡斜树”时，递归深度会达到十万层，这必然导致系统 Stack Overflow (栈溢出) 错误并强制终止程序。

本程序的优势：

迭代算法通过 malloc 手动申请了一块连续数组作为堆栈，内存通常可达数 GB。因此，本程序在时间复杂度维持 O(N)的前提下，消除了深层递归带来的栈溢出隐患，极大地增强了处理极端劣态数据效率。

此外，在解决“两树之和”问题时，也可以选择“不转换数组，直接遍历一棵树并去另一棵树中查找目标差值”的算法。但如果在极端斜树情况下，单次查找的时间复杂度会退化为 O(N)，总时间复杂度将降低至 $O(N_1 \times N_2)$。本程序采用的“先中序展平再双指针”策略 ，用 O(N) 的空间换取了时间，确保了任何输入形态下都能保持 $O(N_1 + N_2)$ 的稳定性能。

4.3 进一步优化的可能性

尽管程序在时间复杂度上已经达到了线性最优，但在空间复杂度上仍有进一

步优化的空间，据查阅相关资料，我获得了一些优化建议：

1. 数据结构表达的优化（从数组索引到纯指针）

在本项目中，我使用了 Input 结构体数组来暂存节点和父子关系，这在处理像 10 万行这种大规模数据时，会额外消耗一块不小的内存。

改进方向：在读取数据时，可以直接根据输入的键值建立节点，并利用直接寻址的方式连接父子关系。

好处：这样可以省去“先存入数组、再遍历数组连边”的中间步骤，减少内存占用，使建树过程更加连贯。

2. 内存分配的批处理

目前程序对每一个树节点都调用了一次 malloc。当节点数量达到 10 万个时，频繁的系统调用会产生一定的开销，且容易造成内存碎片。

改进方向：可以预先申请一块足够大的连续内存，然后通过指针移动来分配节点空间。

好处：这能显著提高大规模数据的处理速度，并方便在程序结束时一次性释放所有内存。

3. 搜索逻辑的提前终止

目前的双指针算法在找到匹配对后会继续搜索直到指针相遇。

改进方向：如果我们在中序遍历时记录下两棵树的最大值和最小值，那么在双指针开始前，就可以先判断目标值 N 是否在范围内（例如 $N < Min_1 + Min_2$ 或 $N > Max_1 + Max_2$）。

好处：对于显然无解的情况，程序可以瞬间返回 false，无需进行完整的线性扫描。

Declaration

I hereby declare that all the work done in this project titled Performance Measurement is of my independent effort.

附录

main.c

```
//A+B Search Tree
/*
核心思路：
1. 读入两棵树（存放在结构体数组中）：
   结构体中存储结点的值和父结点的下标索引
   若父结点下标为 -1，则该结点为根结点，记录下来并最终返回
2. 构建两棵树（CreateTree）：
   遍历所有结点，根据 BST 性质（左小右大）将当前结点连接到父结点上
3. 中序遍历两棵树：
   由于是 BST，中序遍历得到的是严格递增的有序序列
4. 双指针法寻找 A + B = N:
   i 指向序列 A 的头部（最小值），j 指向序列 B 的尾部（最大值）
   若和 < N: i++（增大 A 的值）
   若和 > N: j--（减小 B 的值）
   若和 = N: 输出，并跳过重复值（i++ 且 j--）
5. 先序遍历输出两棵树
*/
#include<stdio.h>
#include<stdlib.h>

#define true 1
#define false 0

typedef int bool;

typedef struct node *Tree;
struct node{
    int key; // 结点存储的数值
    Tree left;  // 左子树指针
    Tree right; // 右子树指针
};

// 辅助结构：用于读入时暂存结点信息和父结点索引
struct input{
    int parent;  // 父结点在数组中的下标
    Tree cell;   // 指向当前结点的指针
};
typedef struct input Input;

//工具函数（堆栈）
```

```
void Push(Tree Stack[],Tree node,int* top){ Stack[++(*top)] = node;} // 入栈
Tree Pop(Tree Stack[],int* top){return Stack[(*top)--];}  // 出栈并返回栈顶元素

Tree CreateTree(int N);  // 根据输入构建二叉搜索树，返回根结点
int* InOrder(Tree T,int N);  // 中序遍历，返回有序数组
void PreOrder(Tree T,int N);  // 先序遍历并输出结点值

int main (){
    // 读入并生成两棵树
    int N1;
    scanf("%d",&N1);
    Tree A = CreateTree(N1);

    int N2;
    scanf("%d",&N2);
    Tree B = CreateTree(N2);

    // 读入目标和 N
    int N;
    scanf("%d",&N);

    // 中序遍历得到两个有序序列
    int *a = InOrder(A,N1);
    int *b = InOrder(B,N2);

    // (Debug 检查数据传输是否成功)打印前三个数
    //printf("Debug A: %d %d %d\n", a[0], a[1], a[2]);
    //printf("Debug B: %d %d %d\n", b[0], b[1], b[2]);

    // 查找 A + B = N 的所有不重复数对
    int i = 0;    // 指向序列 A 开头（最小值）
    int j = N2-1; // 指向序列 B 末尾（最大值）
    bool flag = false; // 标记是否找到至少一个数对
    while(i<N1&&j>=0){
        long long sum = (long long)a[i] + (long long)b[j];  // 防止溢出选择 long long
        if(sum==N) {
            if(!flag){
                printf("true\n");
                flag = true;
            }
            printf("%d = %d + %d\n",N,*(a+i),*(b+j));
```

```
            // 跳过重复的值
            do{i++;}while(i<N1&&a[i]==a[i-1]);
            do{j--;} while(j>=0&&b[j]==b[j+1]);
        }
        // 和太小，需要增大 A 的值
        else if(sum<N) do{i++;}while(i<N1&&a[i]==a[i-1]);
        // 和太大，需要减小 B 的值
        else do{j--;} while(j>=0&&b[j]==b[j+1]);
    }
    // 未找到任何数对，输出 false
    if(!flag) printf("false\n");

    //后序遍历
    PreOrder(A,N1);
    PreOrder(B,N2);

    return 0;
}

// CreateTree
// 输入格式：共 N 行，每行两个整数 key 和 parent_index
// parent_index 为 -1 表示根结点
// 根据 BST 性质（左小右大）自动挂载为父结点的左孩子或右孩子
Tree CreateTree(int N){
    Input *a = (Input *)malloc(sizeof(Input) * N);
    Tree root = NULL;

    // 读取所有结点的值和父结点索引，并分配内存
    for(int i=0;i<N;i++){
        a[i].cell = (Tree)malloc(sizeof(struct node));
        scanf("%d %d",&a[i].cell->key,&a[i].parent);
        a[i].cell->left = a[i].cell->right = NULL;

        // 记录根节点
        if (a[i].parent == -1) root = a[i].cell;
    }
    // 结点连接
    for(int i=0;i<N;i++){
        if(a[i].parent!=-1){
            // 当前结点值小于父结点值 → 左孩子，否则 → 右孩子
            if(a[i].cell->key<a[a[i].parent].cell->key)
a[a[i].parent].cell->left = a[i].cell;
            else a[a[i].parent].cell->right = a[i].cell;
        }
```

```
    }

    //释放空间
    free(a);

    //返回根节点
    return root;
}

// 返回一个包含所有结点值的动态数组，按升序排列

int* InOrder(Tree T,int N){
    //创建堆栈
    Tree *Stack = (Tree *)malloc(sizeof(Tree) * N);
    int top = -1;

    int *result = (int *)malloc(sizeof(int)*N);
    int *s = result; // 用于顺序填充结果数组

    while(top!=-1||T){
        // 一直向左走，将路径上的结点压栈
        while(T){
        Push(Stack,T,&top);
        T = T->left;
        }
        // 弹出栈顶结点并访问
        if(top!=-1){
            Tree node = Pop(Stack,&top);
            *(s++) = node->key;  // 存入结果数组
            T = node->right; // 遍历右子树
        }
    }

    free(Stack);
    return result;
}

// 按先序顺序输出结点值，结点间以空格分隔，末尾换行
void PreOrder(Tree T,int N){
    Tree *Stack = (Tree *)malloc(sizeof(Tree) * N);
    int top = -1;
    bool is_first = true; // 控制输出格式，避免行首多余空格

    while(top!=-1||T){
```

```c
        // 一直向左走，每遇到一个结点就立即输出并压栈
        while(T){
            if(is_first) {
                printf("%d",T->key);
                is_first = false;
            }
            else printf(" %d",T->key);
            Push(Stack,T,&top);
            T = T->left;
        }
        // 弹出结点并转向右子树
        if(top!=-1){
            Tree node = Pop(Stack,&top);
            T = node->right;
        }
    }
    printf("\n");
    free(Stack);
}
```

data_gen.c

```c
/*
    程序功能：生成两棵斜链状二叉搜索树（BST）的测试数据
    输出格式：
       第一行：第一棵树的结点数 N1
       接下来 N1 行：每行两个整数 key 和 parent_index（-1 表示根结点）
       接着一行：第二棵树的结点数 N2
       接下来 N2 行：每行两个整数 key 和 parent_index
       最后一行：目标和 N

       树结构说明：
         两棵树均满足二叉搜索树性质（左小右大）
 */
#include <stdio.h>

int main() {
    int N1 = 100000; // 第一棵树规模
    int N2 = 100000; // 第二棵树规模

    // 第一棵树
    printf("%d\n", N1);
    for (int i = 0; i < N1; i++) {
        // 第一个结点（i=0）为根，parent = -1
        // 后续结点的父结点为前一个结点（i-1）
```

```
        printf("%d %d\n", i * 2, i - 1);
    }

    // 第二棵树
    printf("%d\n", N2);
    for (int i = 0; i < N2; i++) {
        printf("%d %d\n", i * 3, i - 1);
    }

    /// 目标和 target 介于两棵树的最小值和最大值之间
    // 第一棵树最小值 0，最大值 (N1-1)*2
    // 第二棵树最小值 0，最大值 (N2-1)*3
    printf("%d\n", 50000);

    return 0;
}
```